Grid Features Validation Packet -- DFJ1K3

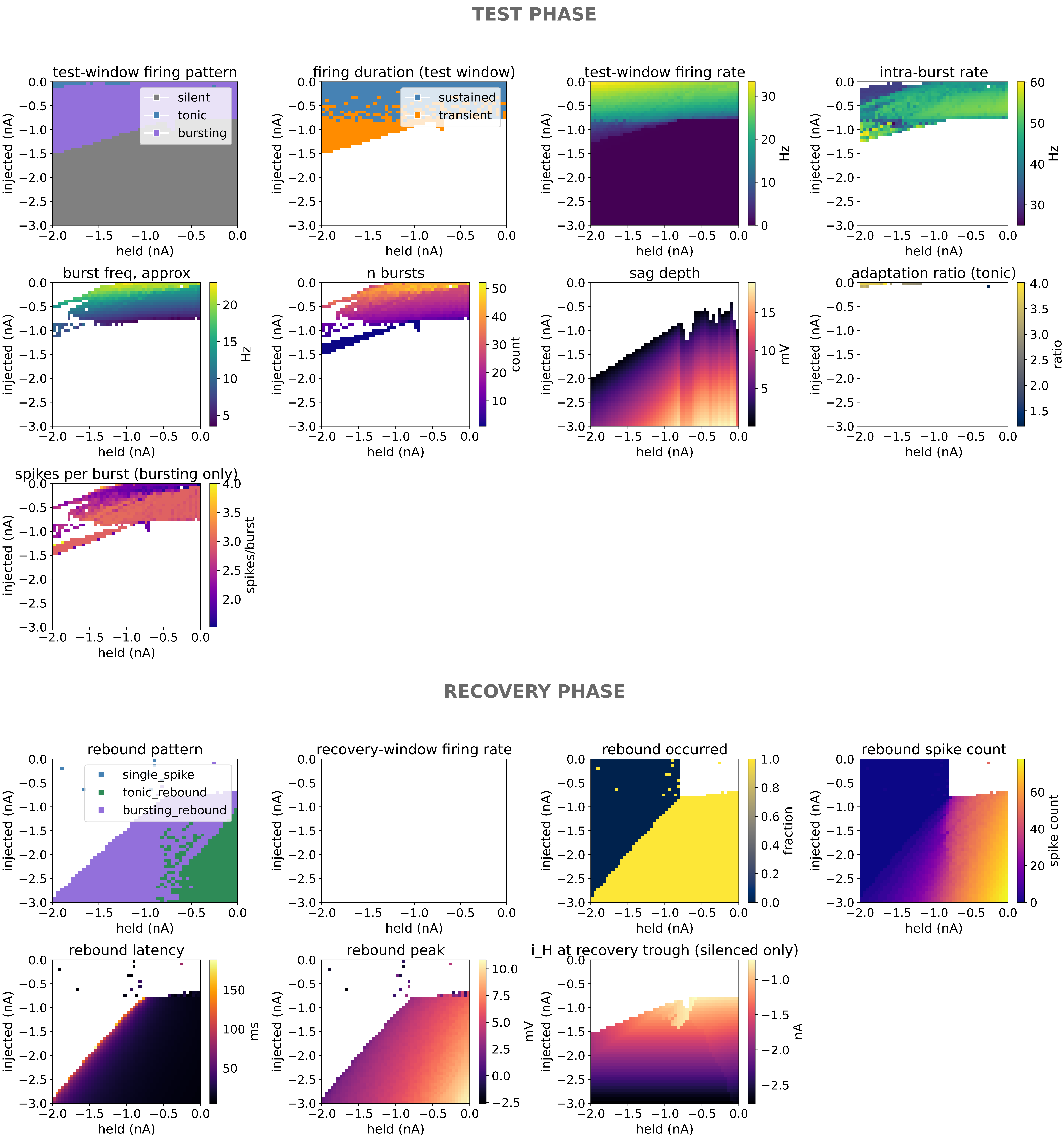
Every trace in this packet is simulated output from a fitted, multi-conductance single-compartment model of this cell, integrated deterministically -- not a raw current-clamp recording. What is being evaluated here is the feature-extraction methodology applied on top of that model: spike detection, burst/tonic classification, post-inhibitory rebound detection, sag depth, and spike-frequency adaptation. Each figure below reruns the actual classification algorithm on a real simulated trace and marks what it found directly on the trace.

```
Silencing threshold: -2.00 nA   |   2500 points sampled across the current grid
Burstiness index: 0.335
Firing-rate/current slope: 25.73 Hz/nA (R^2=0.924)
```

Contents:


```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl + cell_grid_features.pkl
```

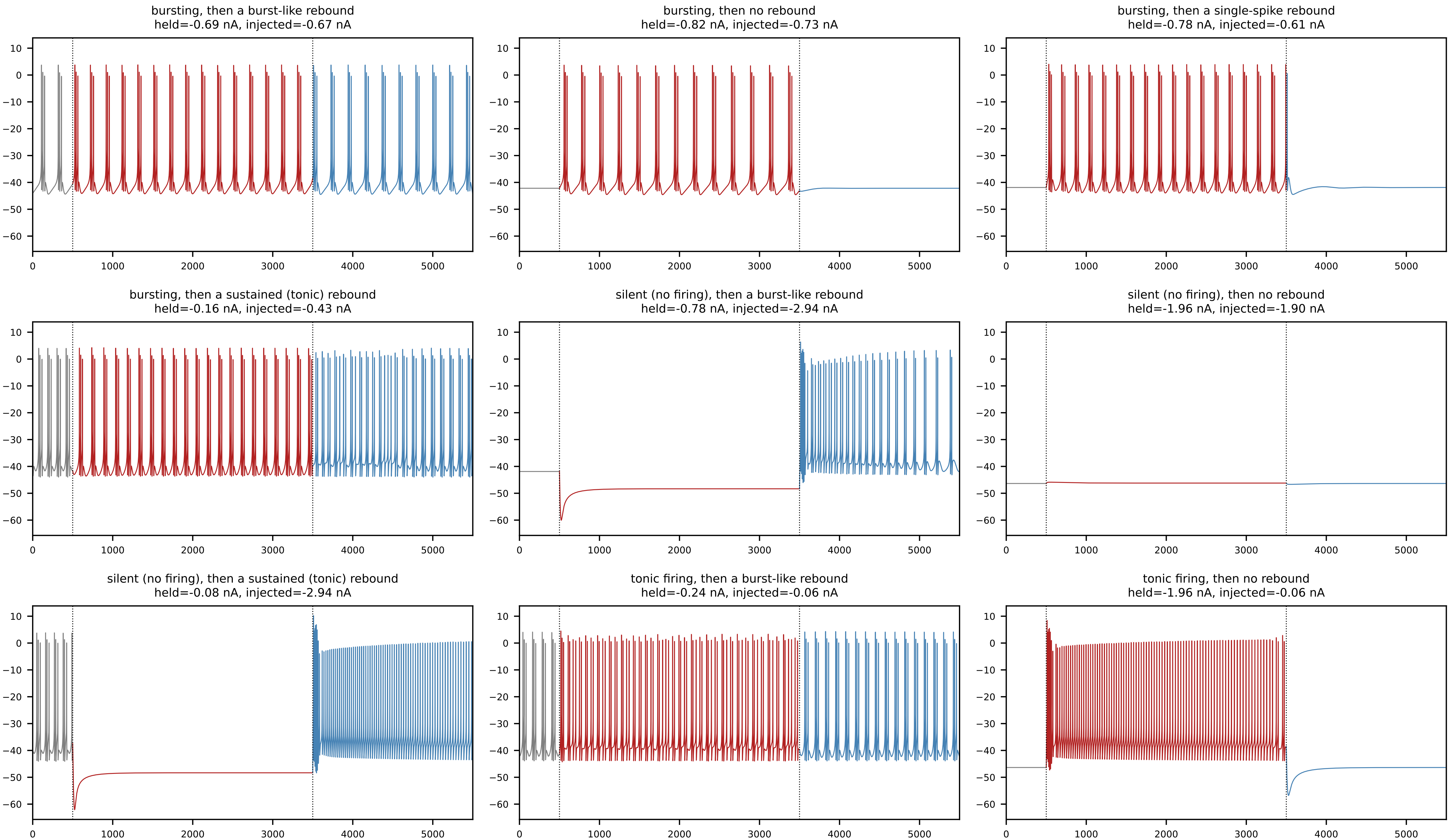
1. Feature overview: DFJ1K3's full current grid



Every panel below is computed from the same 2500-point grid of held and injected current levels: which pattern each point fires in and whether it rebounds after release from hyperpolarization (top row), firing rate and burst structure (second row), rebound spike count, latency, and peak voltage (third row), and sag depth, adaptation, and burst size (bottom row). Two single-number summaries condense the whole grid: a burstiness index of 0.335 (the fraction of points classified bursting) and a firing-rate-vs-current slope of 25.73 Hz/nA (R^2=0.924). The rest of this packet does not add new results -- it re-derives individual pieces of this same grid from real simulated traces so each one can be checked by eye against the panel it feeds.

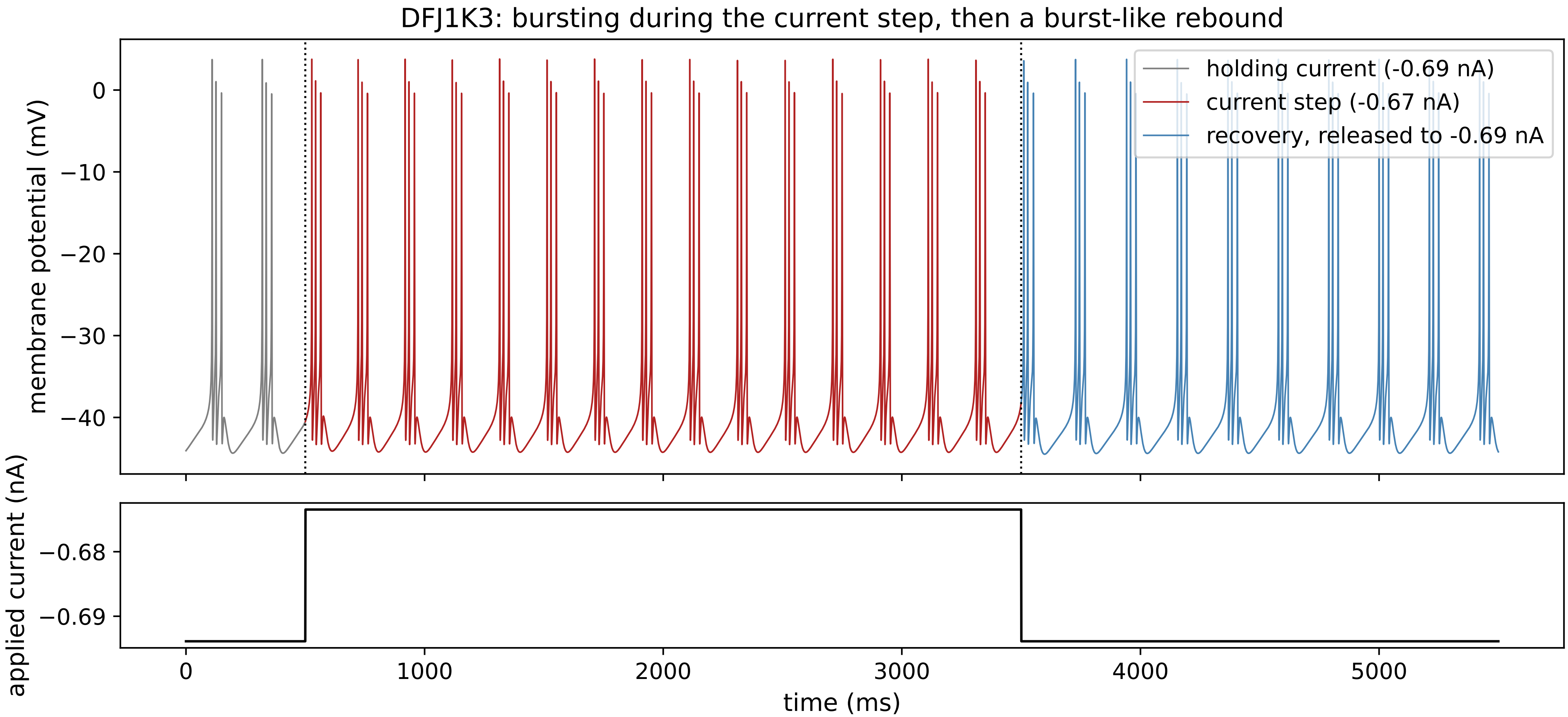
```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> cell_grid_features.pkl (run_held_injected_grid.build_cell_grid_features_fig(), no resimulation)
```

2. Representative traces

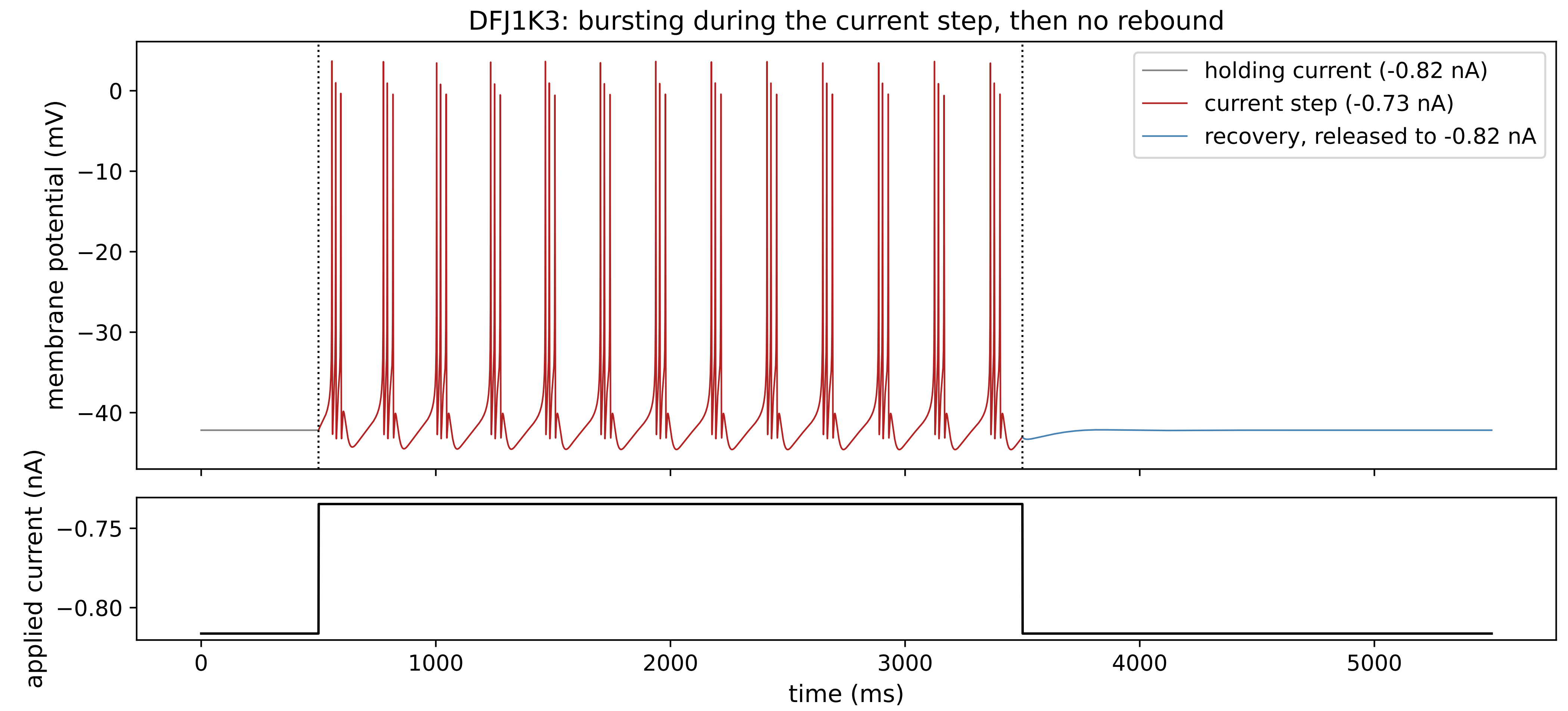


One representative trace for each distinct combination of response-to-current-step pattern and rebound pattern observed across this cell's full 2500-point grid of held and injected current levels: gray is the held baseline before the current step, red is the response during the current step, and blue is the recovery period after release back to the held level. Each combination is represented by its most clearly-expressed example among the grid points that produced it. Every panel shares the same voltage (-66 to 14 mV) and time (0-5500 ms) axes so amplitude and duration are directly comparable by eye -- without this, a small subthreshold wobble and a full-amplitude spike train would each be independently rescaled to fill the same panel and look equally prominent.

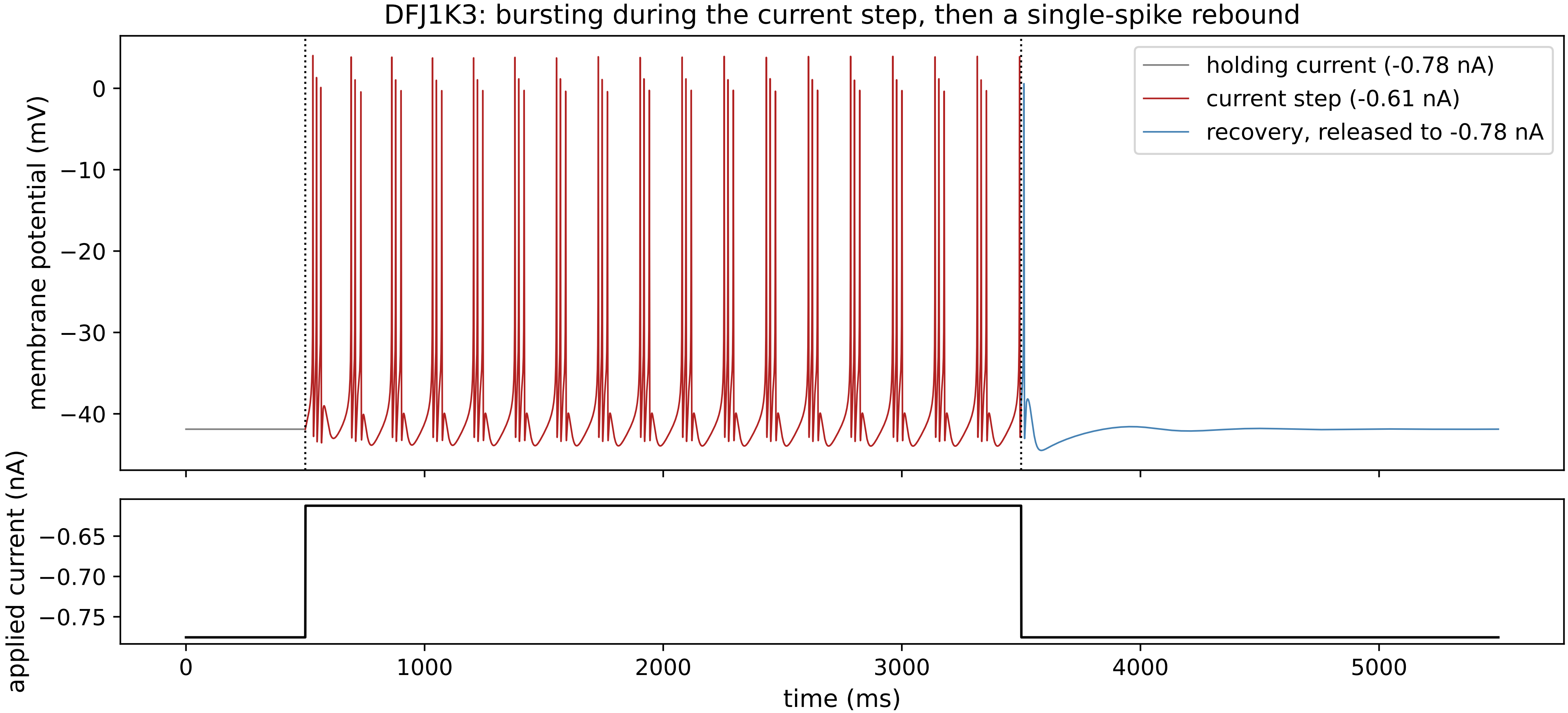
```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point()
```



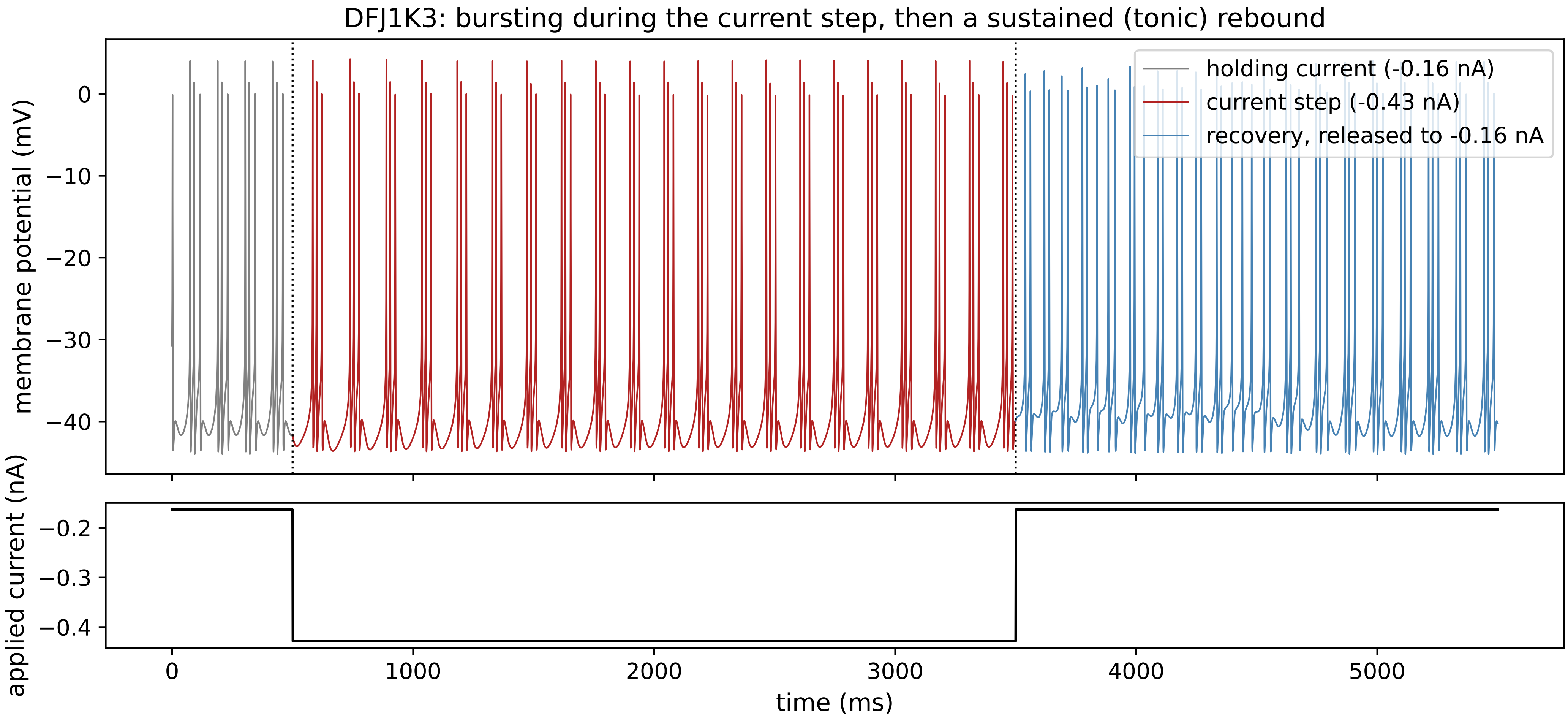
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



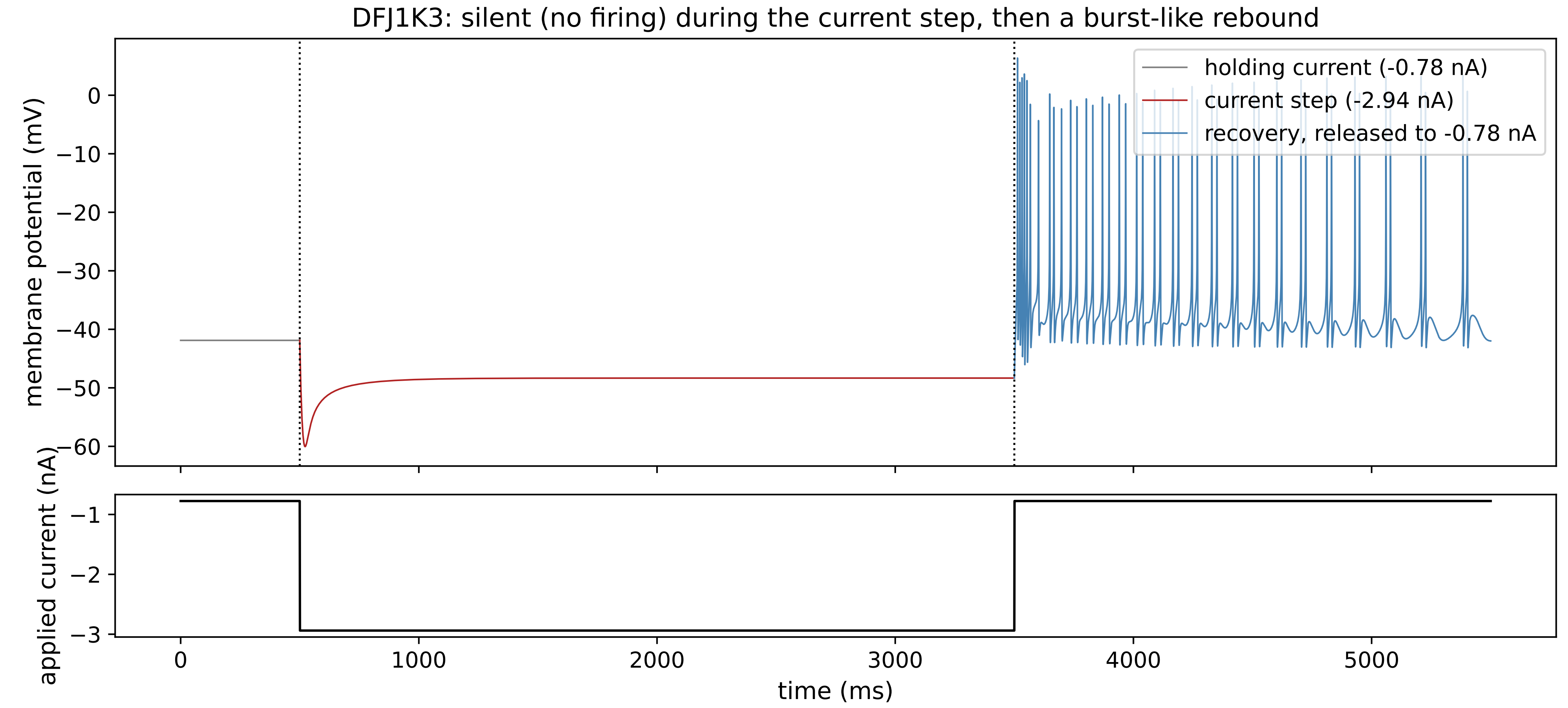
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



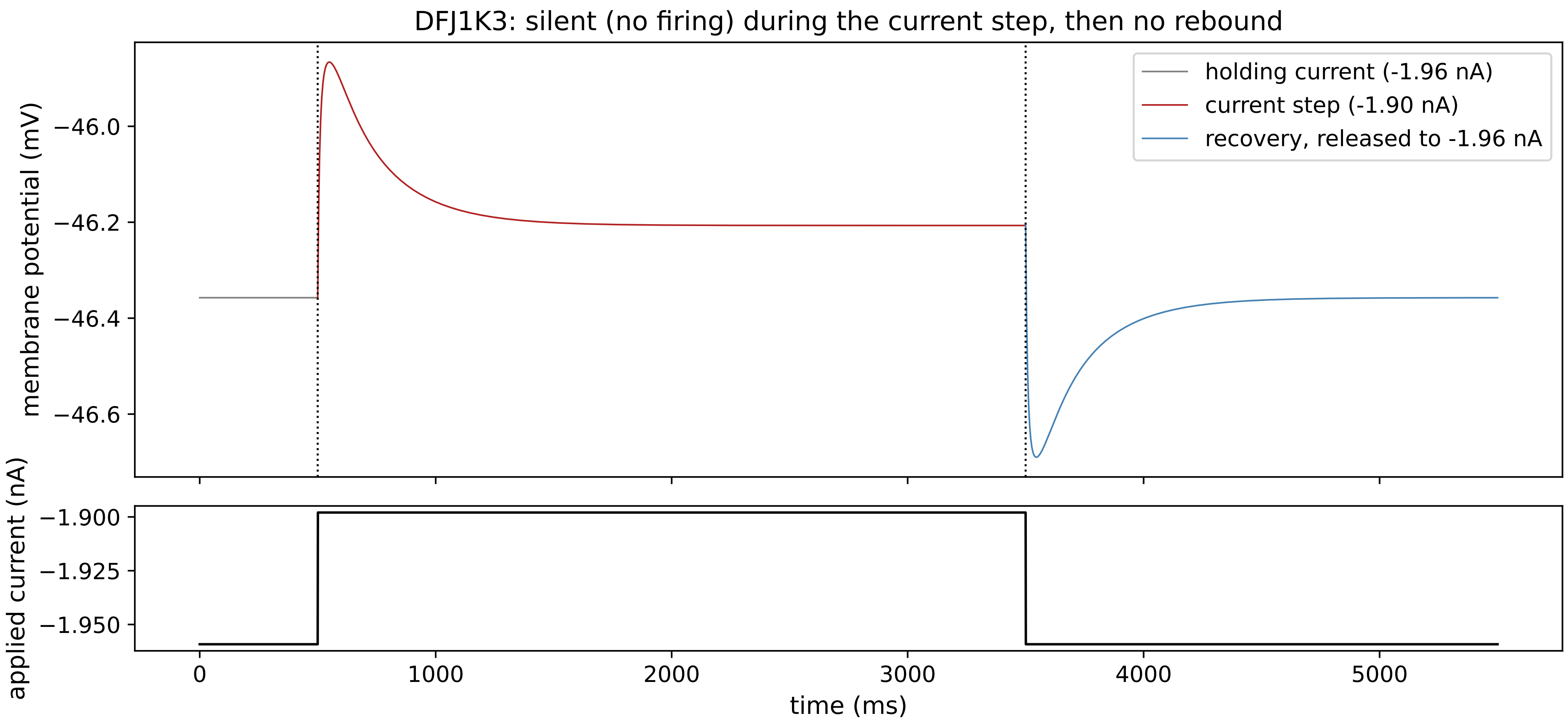
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



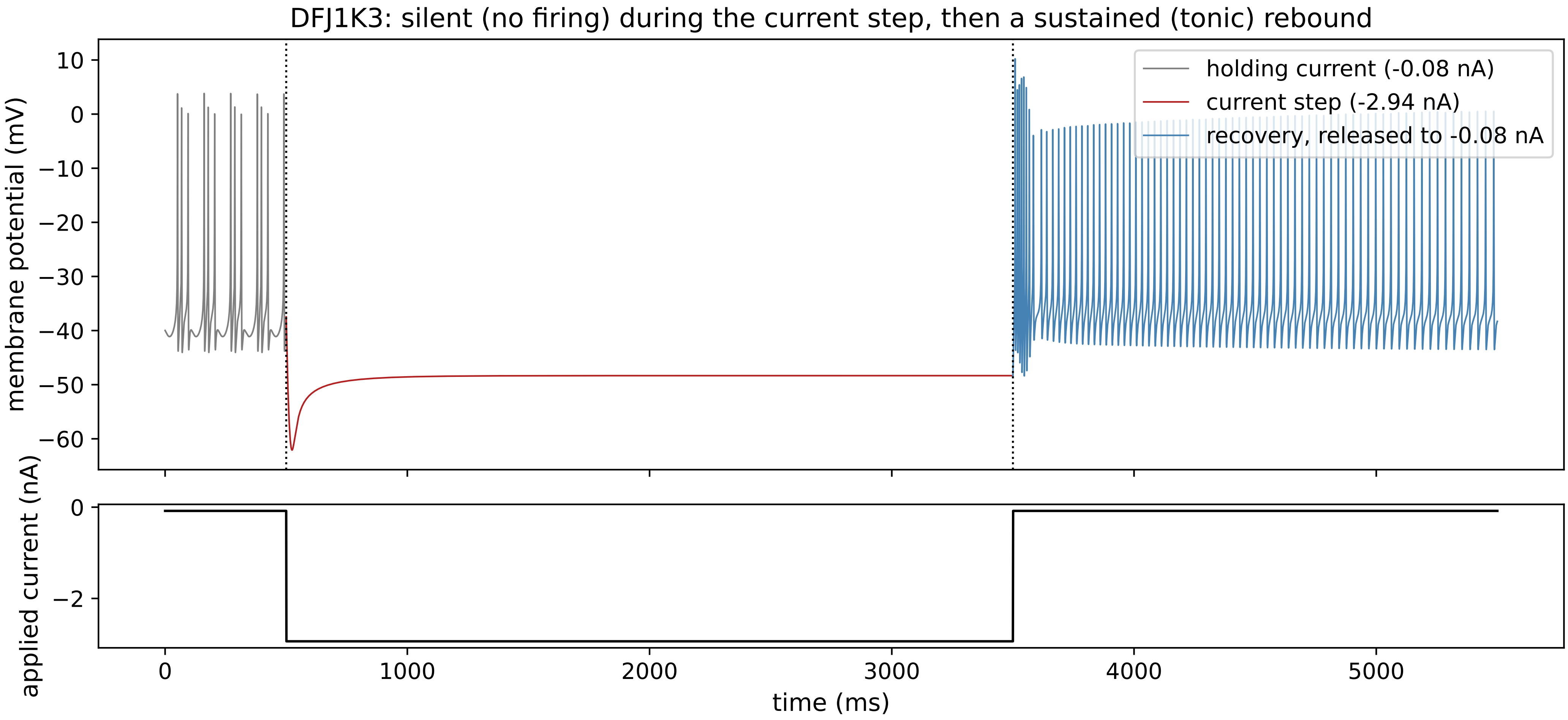

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()
```



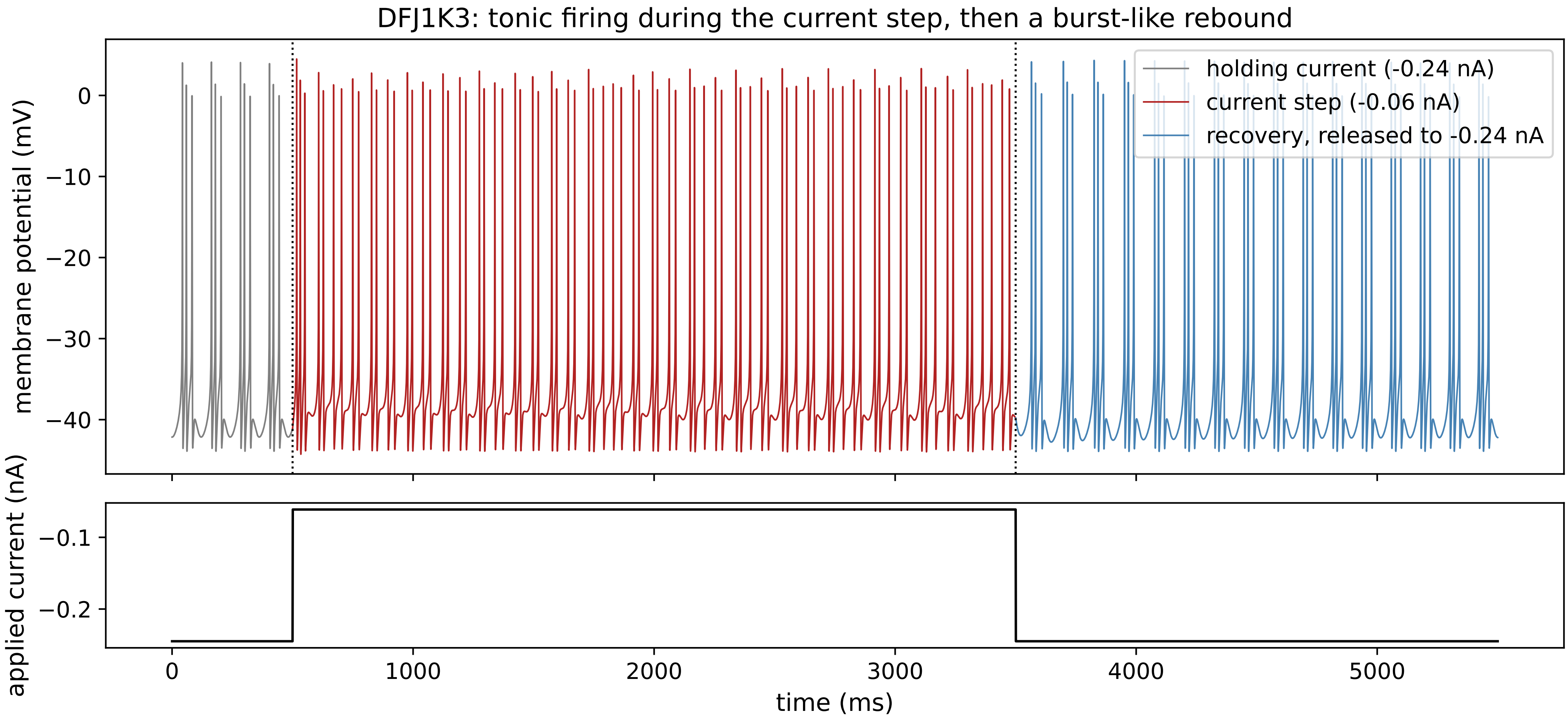
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



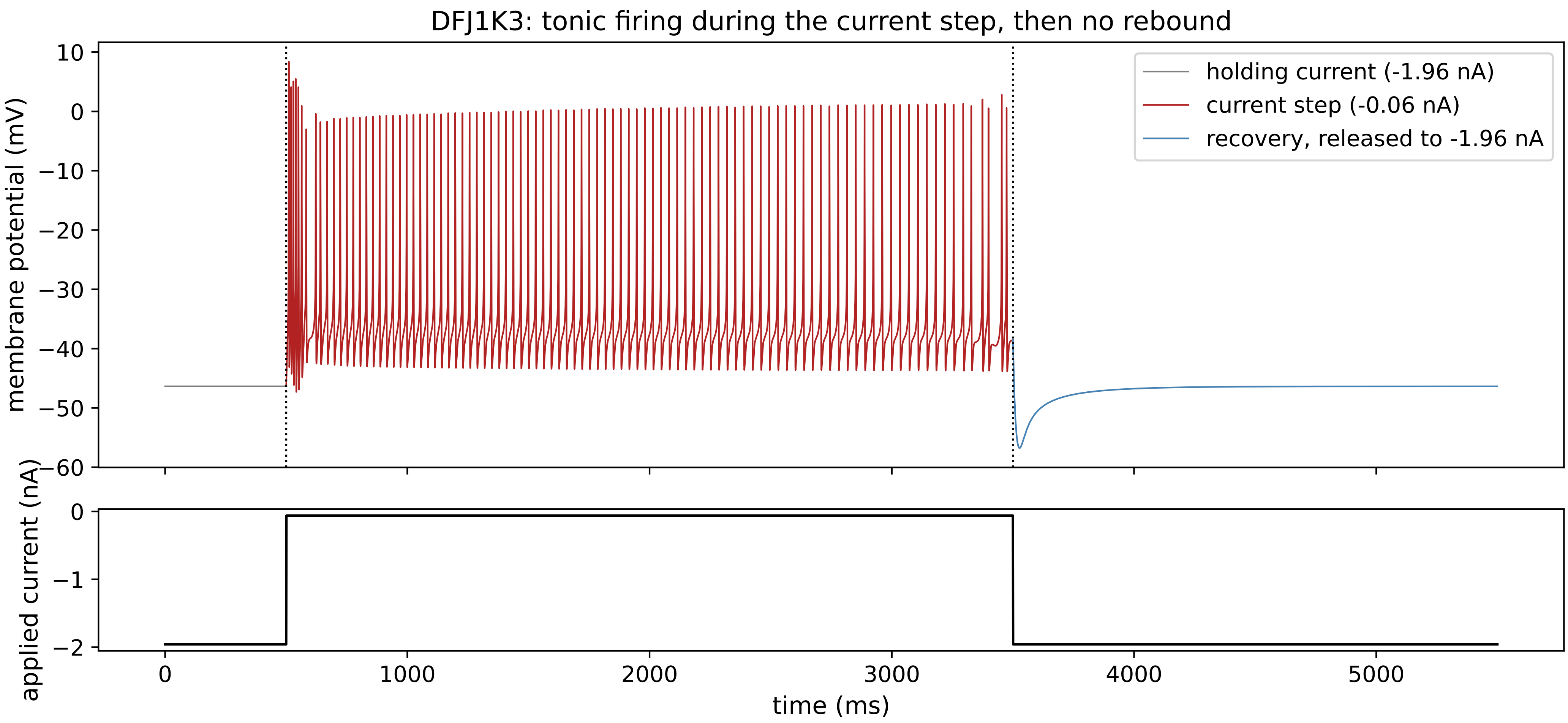
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

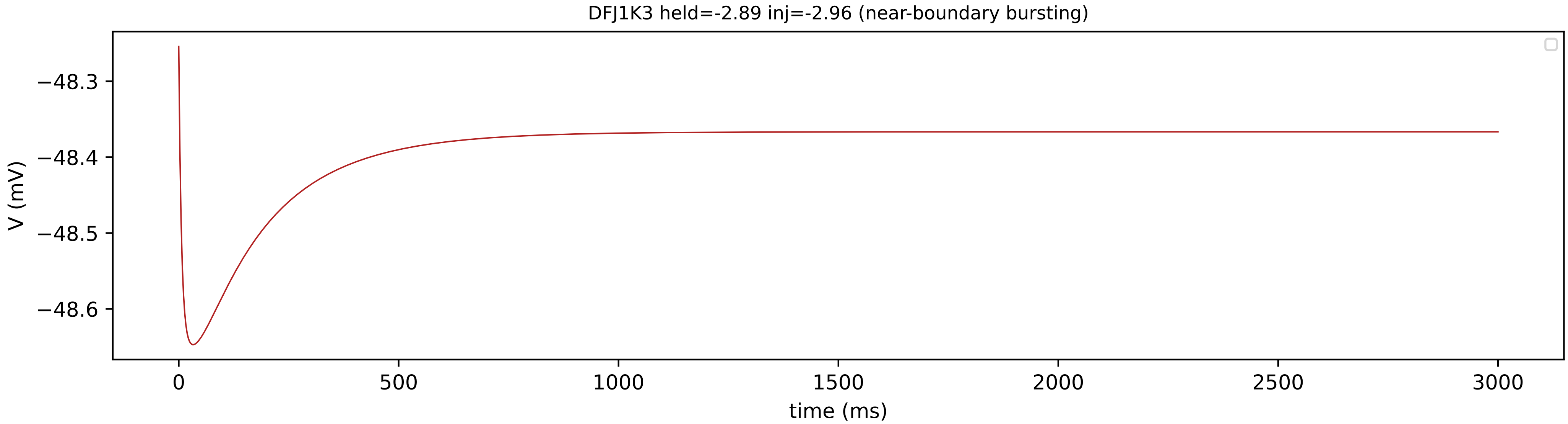


python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

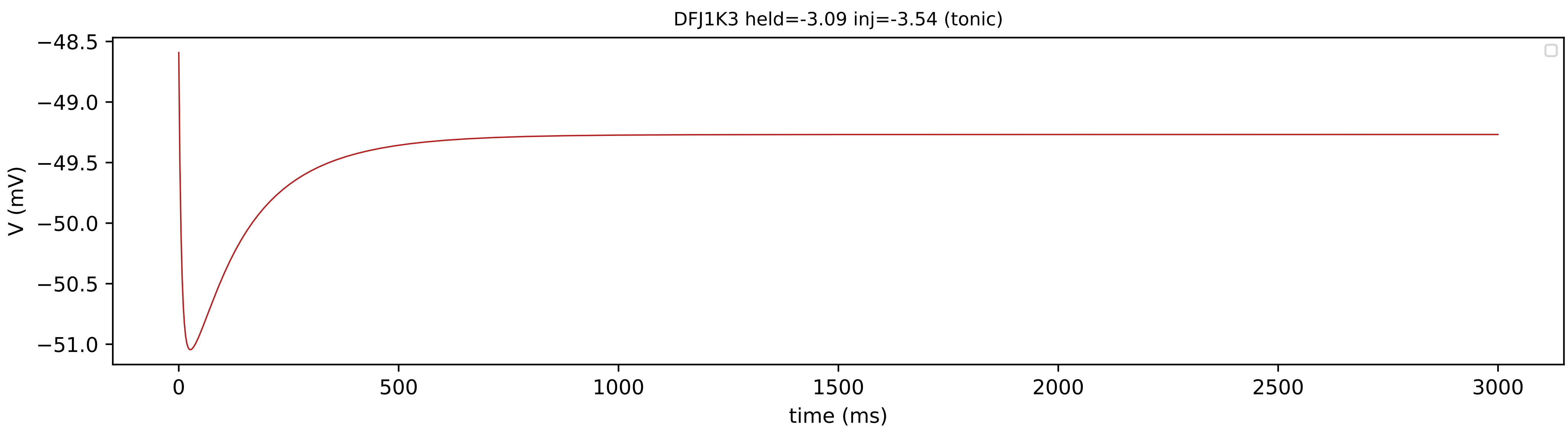


python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

3. Spike detection

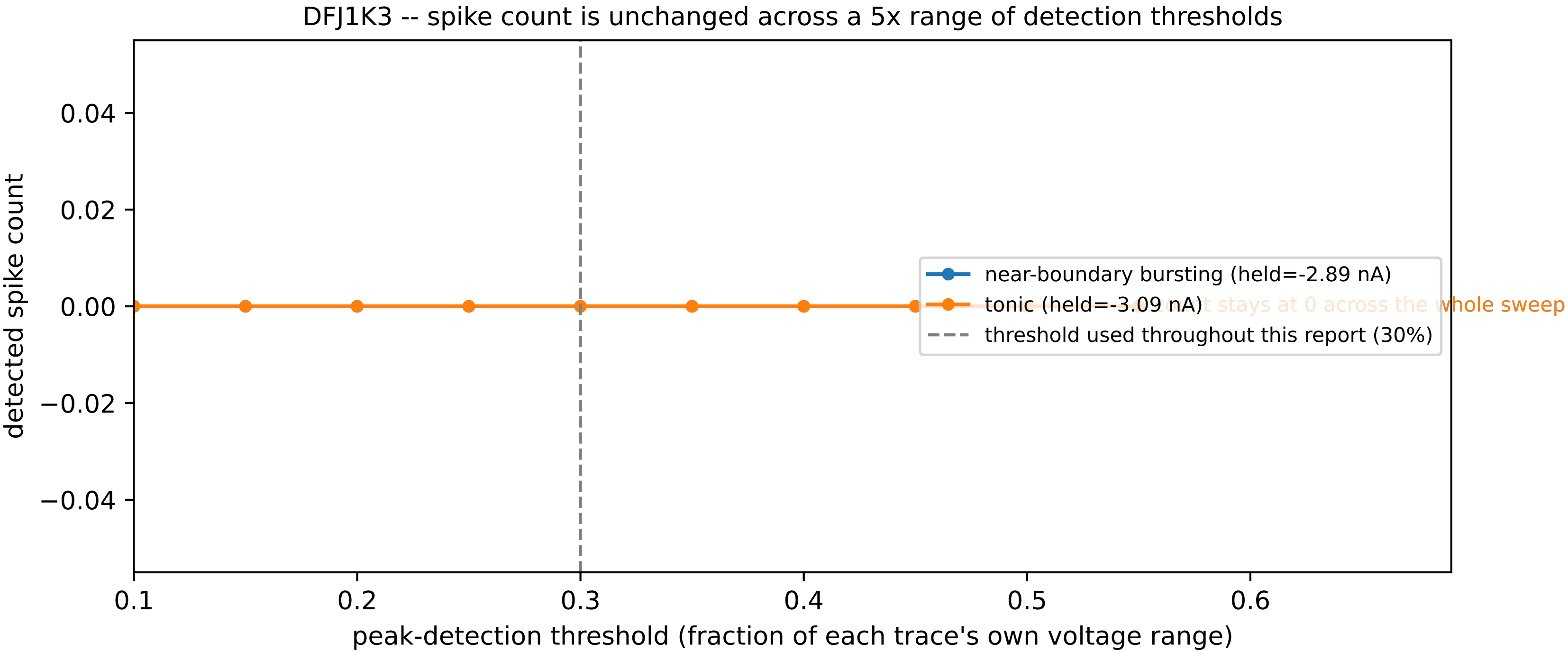


No spikes were detected: the voltage stayed within a 0.39 mV range for the whole window, below the 1.0 mV range required even to attempt peak detection, so this window is classified as electrically silent.



python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> trace_annotations.mark_spikes() (scipy.signal.find_peaks, same call find_silencing_threshold.compute_isis_ms uses)

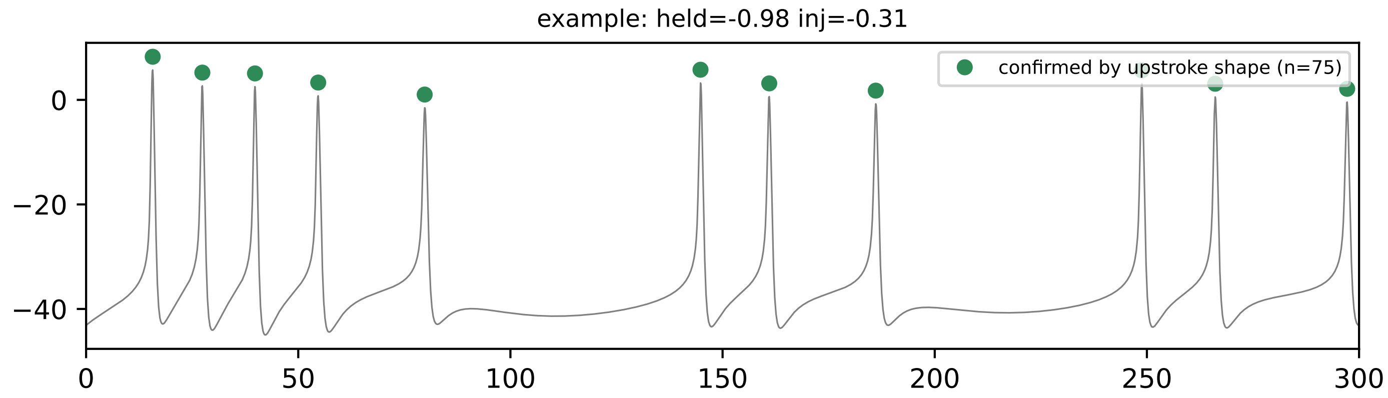
4. Spike-detection threshold sensitivity

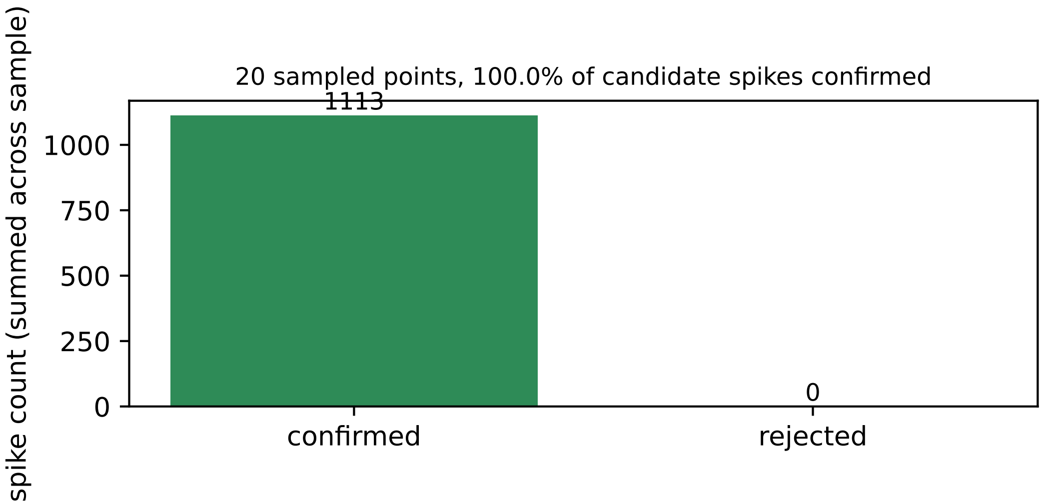


Every spike count, interspike interval, and burst/tonic classification in this report depends on one detection threshold: a candidate peak must rise at least 30% of the trace's own voltage range above its surroundings (or 15 mV, whichever is larger, so a tiny subthreshold wobble in a low-amplitude trace cannot be counted as a spike). This sweeps that threshold from 10% to 55% on real, large-amplitude spiking traces: a flat plateau around the value used throughout this report shows the reported spike counts are insensitive to the exact threshold chosen.

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> scipy.signal.find_peaks swept over PROMINENCE_FRACTION
```

5. Independent cross-check of spike detection by upstroke shape



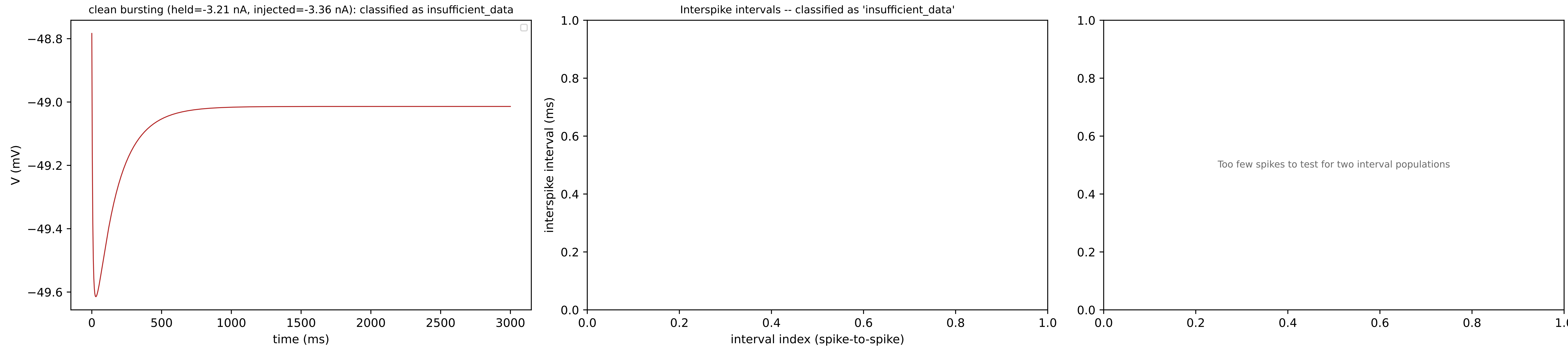


As an independent check, each candidate spike was also required to show a fast upstroke -- a rate of voltage rise of at least 10 mV/ms within 20 ms before the peak (the convention used by Bean, 2007) -- rather than amplitude alone. 75 of 75 candidate spike(s) were confirmed by this stricter test; the two methods agree on every spike in this window.

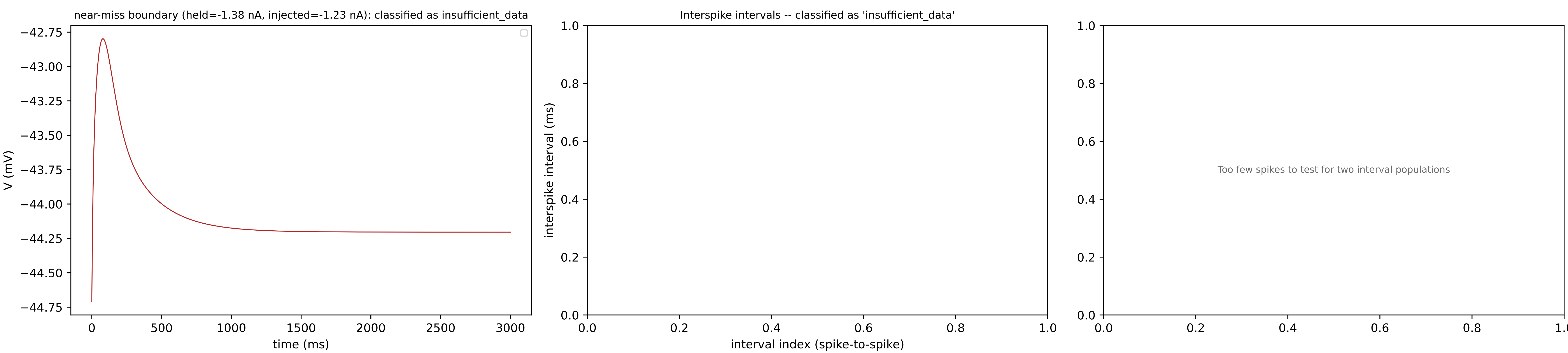
As an independent check on a random sample of 20 grid points from this cell, every spike detected by amplitude alone was additionally required to show a fast upstroke in voltage (rather than amplitude alone) before being counted, an independent detection criterion not used for classification elsewhere in this report. Agreement close to 100% shows that the simpler amplitude-based detector used throughout is not missing any spikes that a shape-based check would catch.

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.detect_spikes_dvdt_confirmed()
```

6. Tonic / bursting / silent classification



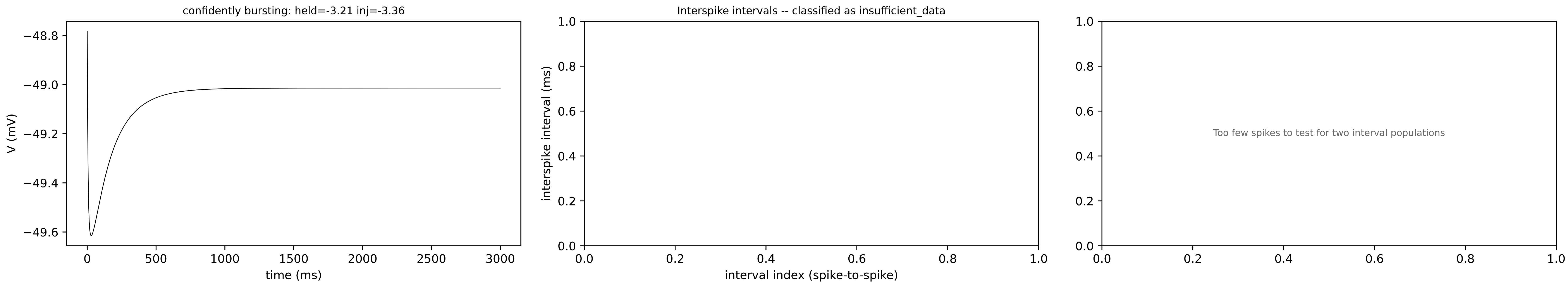
This window had 0 spike(s), giving 0 interspike interval(s) -- fewer than the 6 needed to statistically test whether intervals fall into two separate populations (short, within-burst gaps vs. long, between-burst gaps), so the window defaults to 'insufficient_data'.



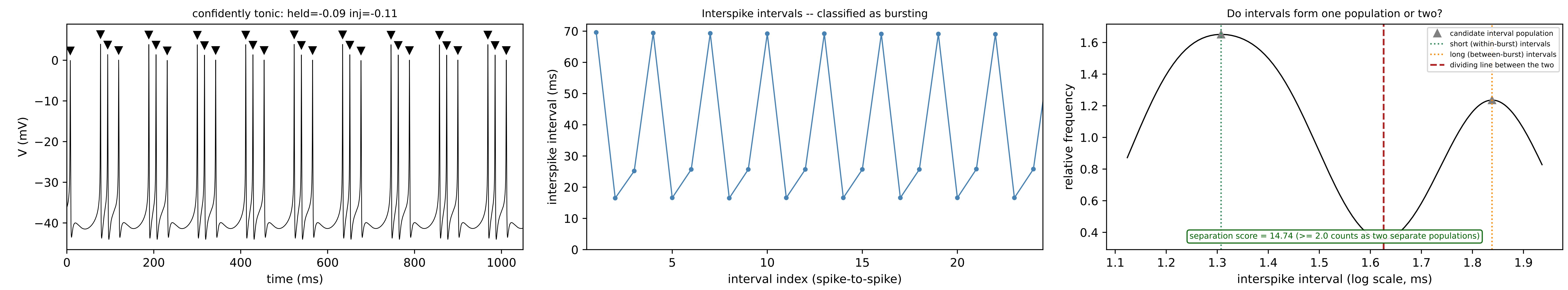
This window had 0 spike(s), giving 0 interspike interval(s) -- fewer than the 6 needed to statistically test whether intervals fall into two separate populations (short, within-burst gaps vs. long, between-burst gaps), so the window defaults to 'insufficient_data'.

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.classify_burst_pattern()
```

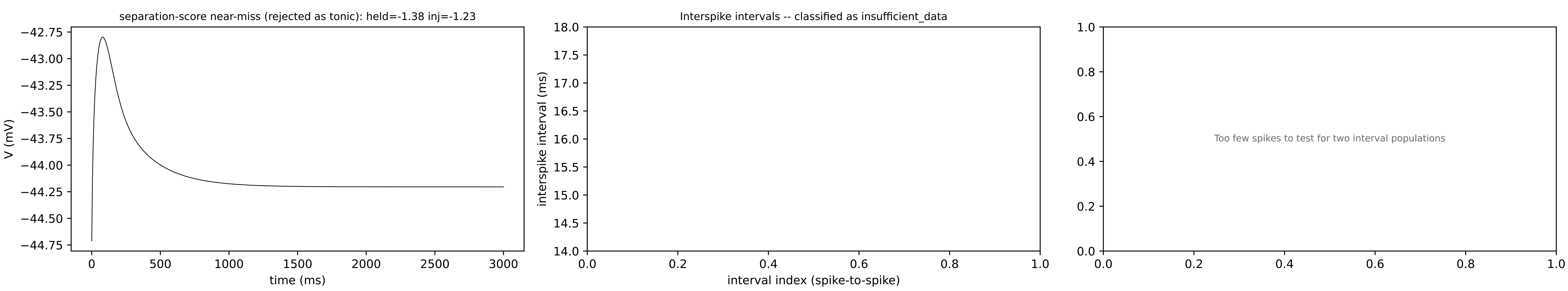
7. Bimodality test: within-burst vs. between-burst ISIs



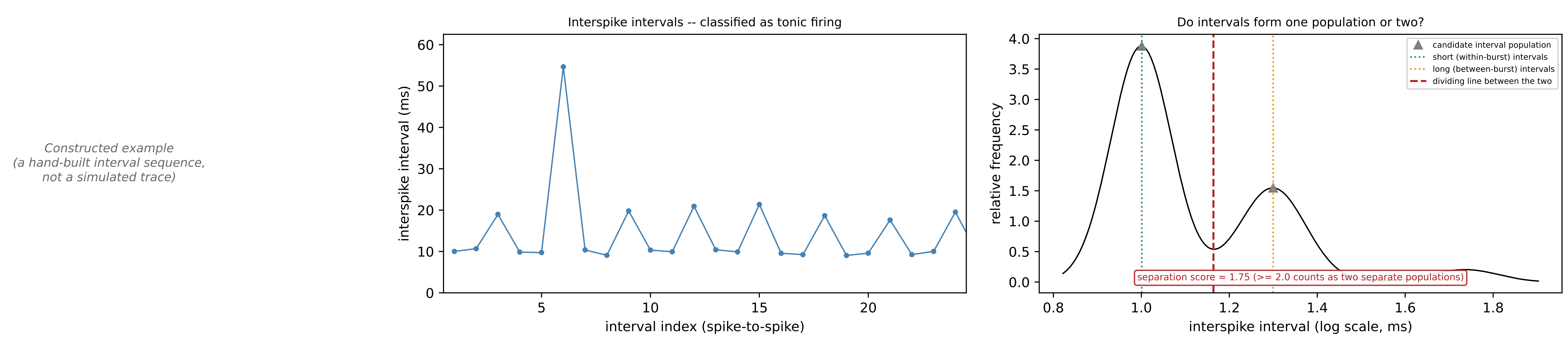
Simulated DFJ1K3 trace. This window had 0 spike(s), giving 0 interspike interval(s) -- fewer than the 6 needed to statistically test whether intervals fall into two separate populations (short, within-burst gaps vs. long, between-burst gaps), so the window defaults to 'insufficient_data'.



Simulated DFJ1K3 trace. This window contained 81 spikes, giving 80 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 21.1 ms and a long (between-burst) interval of 69.0 ms -- a 3.27-fold difference, above the 1.5-fold difference required to treat them as distinct. On average, 2.89 spikes fall within each burst (a real burst requires at least 1.5, ruling out what would otherwise just be occasional missed spikes in an otherwise tonic train). A statistical separation score (Ashman's D) of 14.74 quantifies how cleanly the short and long intervals separate into two distinct populations (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'bursting', with a separation score of 14.74.



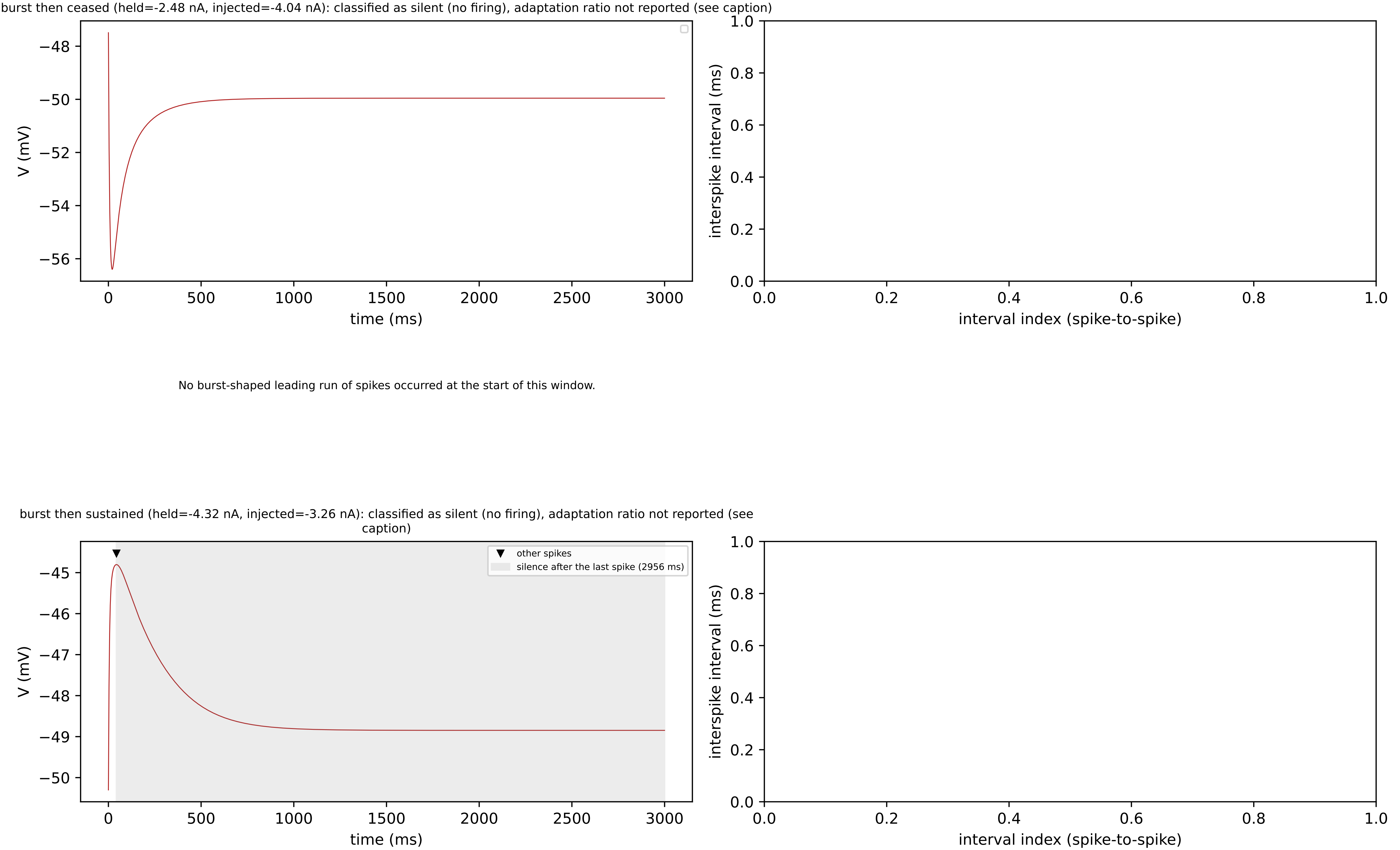
Simulated DFJ1K3 trace. This window had 0 spike(s), giving 0 interspike interval(s) -- fewer than the 6 needed to statistically test whether intervals fall into two separate populations (short, within-burst gaps vs. long, between-burst gaps), so the window defaults to 'insufficient_data'.



This example is constructed, not an observed point from DFJ1K3: it illustrates a case where the short and long interval populations pass the simpler gates but remain too statistically overlapping to count as separate, a case this cell's own data has not been observed to produce. This window contained 180 spikes, giving 179 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 10.0 ms and a long (between-burst) interval of 24.0 ms -- a 2.40-fold difference, above the 1.5-fold difference required to treat them as distinct. On average, 3.00 spikes fall within each burst (a real burst requires at least 1.5, ruling out what would otherwise just be occasional missed spikes in an otherwise tonic train). A statistical separation score (Ashman's D) of 1.75 quantifies how cleanly the short and long intervals separate into two distinct populations (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'tonic', with a separation score of 1.75.

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.classify_burst_pattern() (4th row: constructed synthetic ISIs, no resimulation)
```

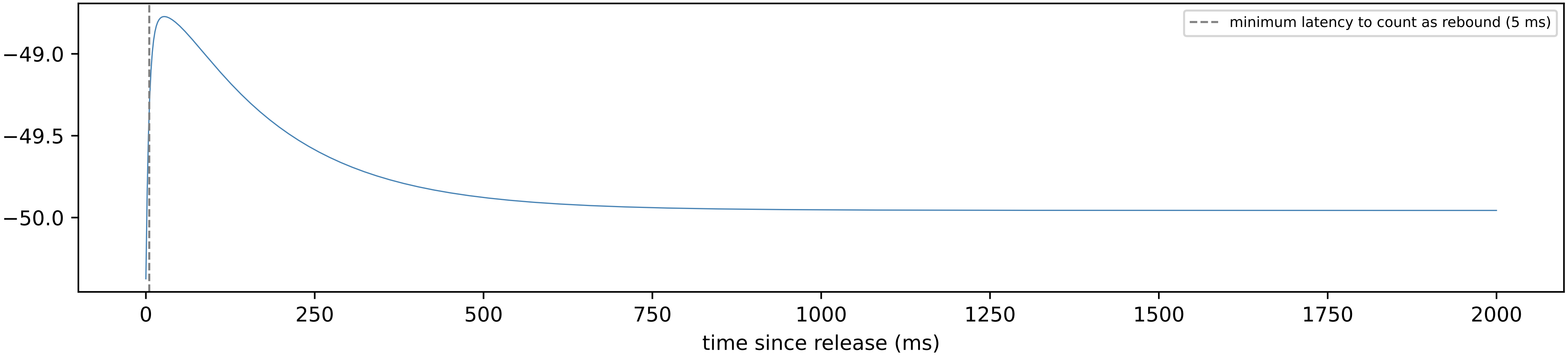
8. Burst-onset-then-silence detection




```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.detect_onset_burst()
```

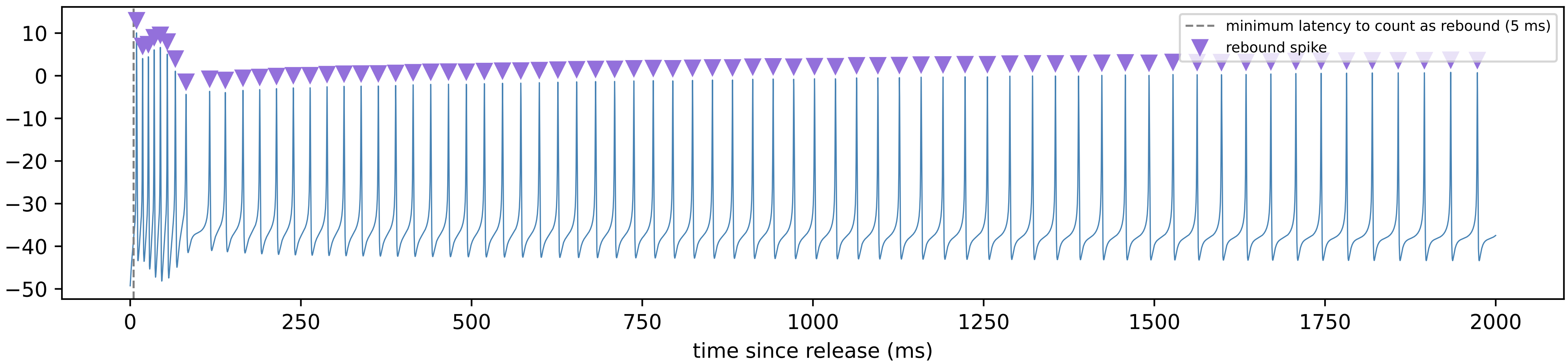
9. Post-inhibitory rebound detection

single-spike rebound: held=-4.04 inj=-4.38



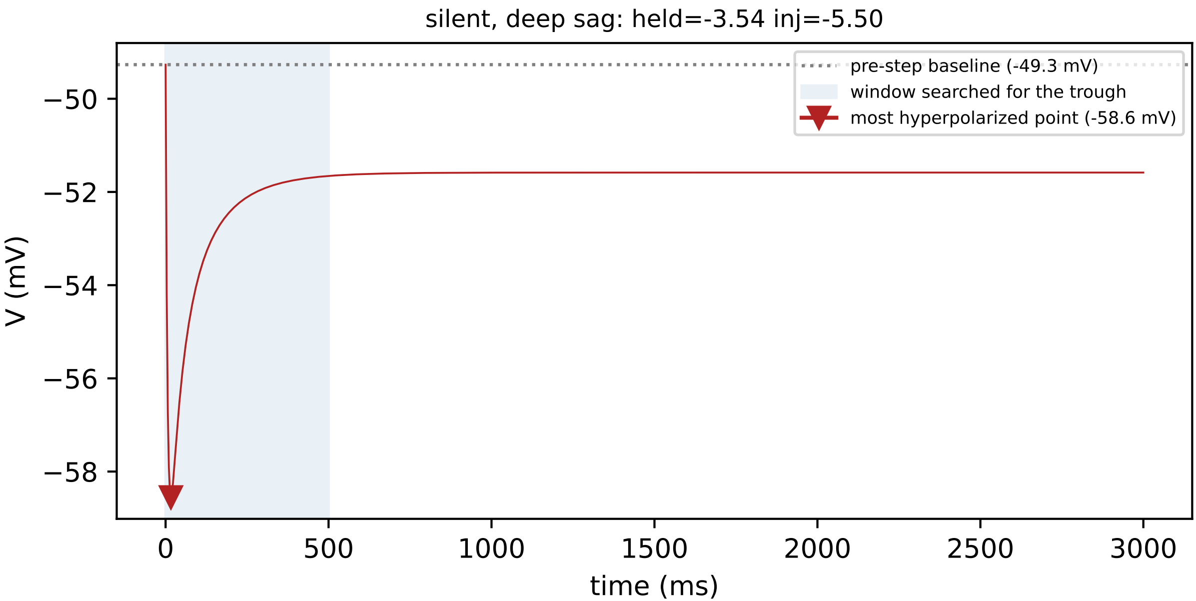
0 spike(s) occurred at or after 5 ms post-release and so count as post-inhibitory rebound. This cell did not show a rebound response.

sustained (tonic) rebound: held=-0.32 inj=-3.54



python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> trace_annotations.mark_rebound_window() (reproduces run_held_injected_grid.run_test_and_recovery's rebound-peak logic)

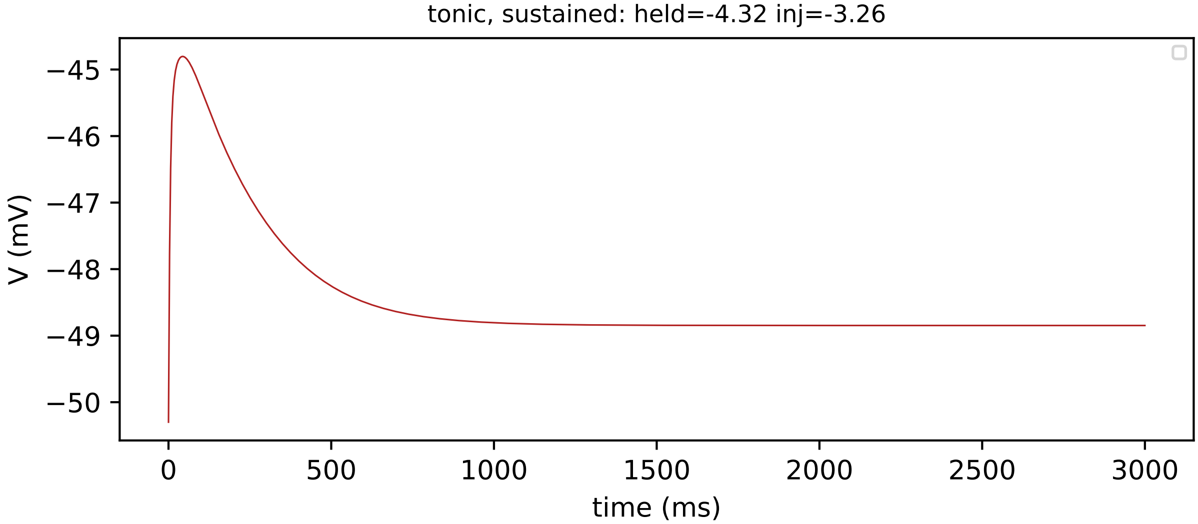
10. Sag depth

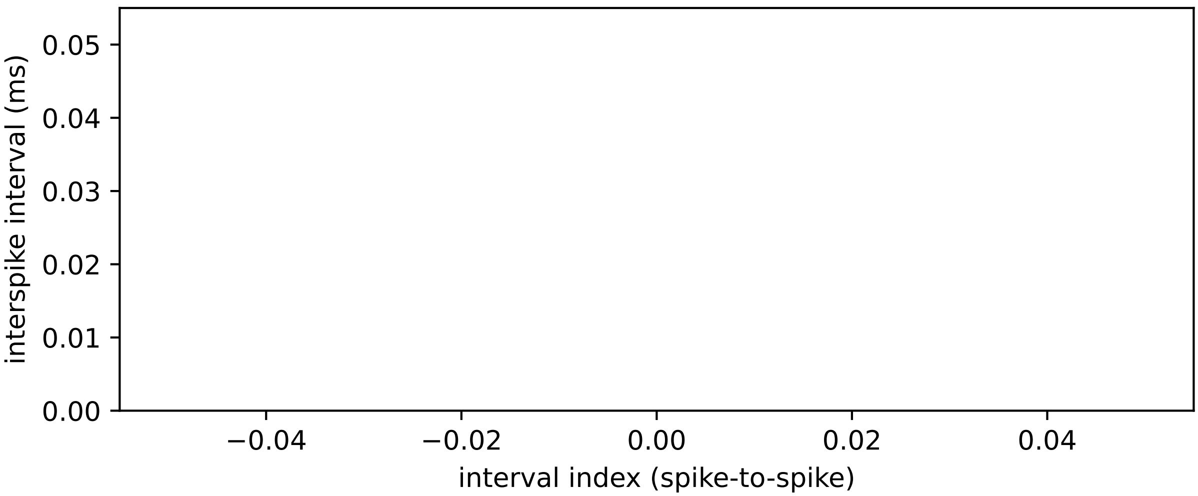


Sag depth is the difference between the voltage just before the current step (-49.3 mV) and the most hyperpolarized point reached afterward (-58.6 mV), giving 9.3 mV of sag -- a hallmark of the hyperpolarization-activated current (Ih) partially repolarizing the cell during the step. The trough was found by searching the full 500 ms window (no spike occurred before it ended).

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_pre_spike_sag_trough()
```

11. Spike-frequency adaptation

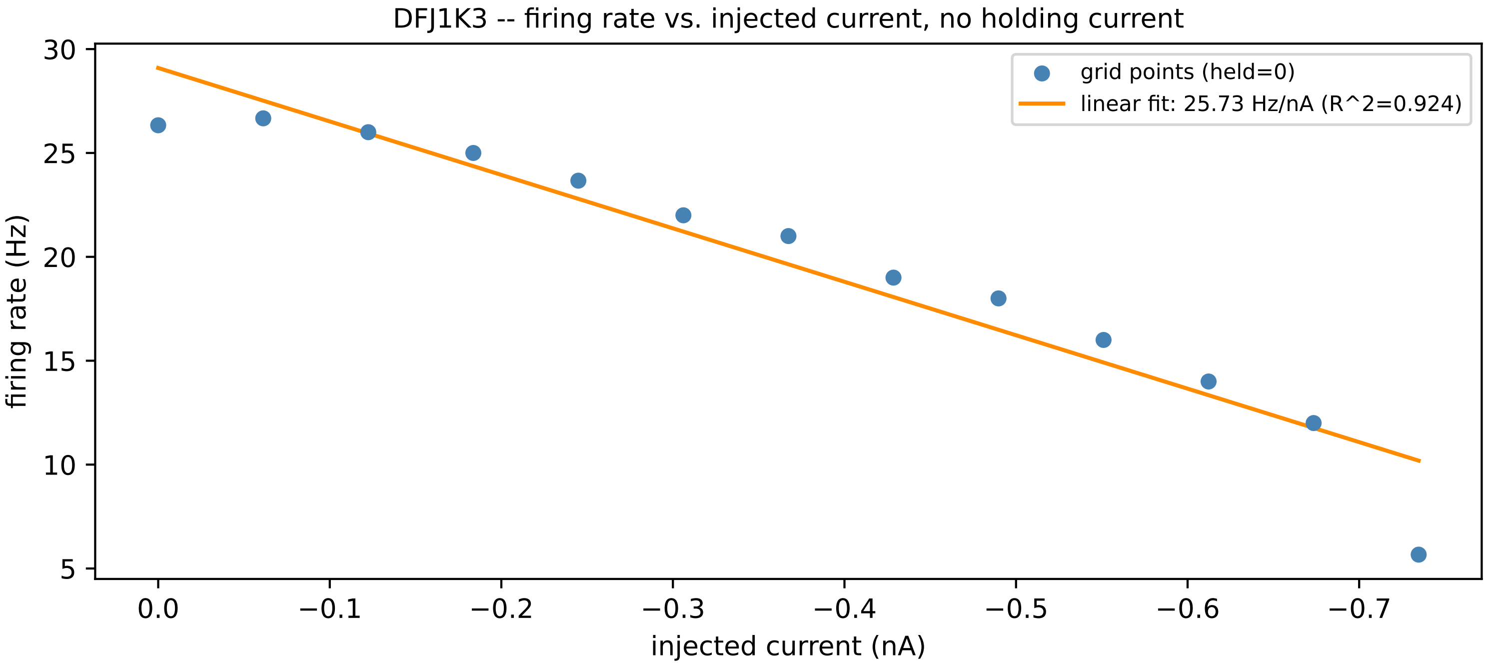




Spike-frequency adaptation was not assessed: this window contained only 0 interspike interval(s), fewer than the 6 needed to compare non-overlapping groups of intervals at the start and end of firing.

python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_adaptation_ratio()

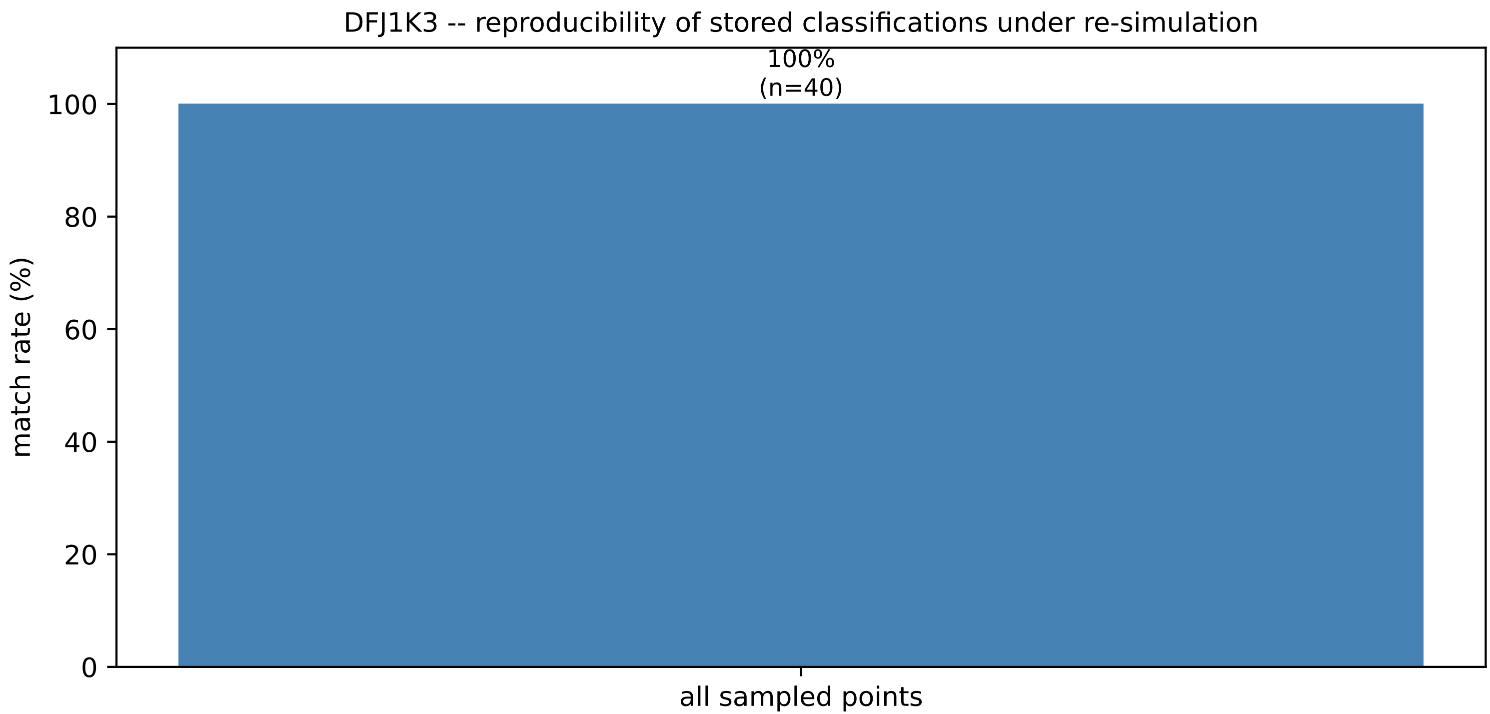
12. Firing rate vs. injected current



13 non-silent points with a clearly-classified firing pattern, sampled across injected current with no holding current applied (points still below firing threshold, which would otherwise bias the linear fit toward zero, are excluded). This firing-rate/current slope is one of the features used to characterize this cell in the cross-cell comparison; here it is plotted directly against the data it was fit to.

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MX2C4S
source: cell_held_injected_grid.pkl -> cell_grid_features.pkl (extract_grid_features.compute_fi_slope())
```

13. Reproducibility check



40 random points from this cell's grid were re-simulated independently and compared against the classification stored from the original sweep. Match rate: 100%. Except at zero holding current, where the cell's own steady-state limit cycle is reused exactly, each holding-current level is reached by integrating forward from whichever already-settled level is numerically nearest -- this model is known to show path-dependent (hysteretic) settling near dynamical transitions, so a re-simulation that approaches a borderline point along a different path than the original sweep can occasionally land on a different classification right at that boundary.

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> cached test_pattern/rebound_pattern vs. fresh resimulate_point() results
```